



Department of Electrical and Information Engineering
University of Ruhuna

Mini Project
Big Data Analytics
EC8203

Voltstream

A Lambda-Architecture Data Platform for
Smart Grid Monitoring and Billing

Use Case 3: Smart Grid Energy Monitoring & Billing

EG/2021/4511	Fernando S.A.U.
EG/2021/4516	Fonseka C. M.
EG/2021/4525	Gunarathna P.M.S.C.B.

Computer Engineering (CE)

September 2026

Repository: <https://github.com/Malee-Fonseka/voltstream>

Contents

1	Use Case and Requirements	2
1.1	Scope and Domain Problem	2
1.2	Interpreted Business Requirements	2
1.3	Functional and Non-Functional Requirements	2
1.4	Simulated Clock	3
2	Architecture Decision: Lambda versus Kappa	3
3	Solution Architecture	5
4	Technology Stack and Justification	6
5	Implementation	8
5.1	Deployment	8
5.2	Data Sources and Ingestion	8
5.3	Processing	9
5.4	Storage and Serving	10
5.5	Code Quality and Testing	10
6	Observability Design	10
7	Results	12
7.1	Real-Time Zone Monitoring	12
7.2	Daily Billing and Solar Report	13
7.3	Provisional-to-Final Transition	13
7.4	Speed-versus-Batch Reconciliation	14
7.5	Observability in Operation	14
7.6	Resilience and Re-billing	15
7.7	Measured Latency	16
8	Discussion, Limitations and Production Scale	16
9	Conclusion	17
	References	17
	Appendix A: Assumptions and Simplifications	18
	Appendix B: Reproducing the Results	18
	Appendix C: Statement of Individual Contributions	18

1. Use Case and Requirements

1.1. Scope and Domain Problem

This report describes **voltstream**, the data platform we built for Use Case 3 of the brief, *Smart Grid Energy Monitoring and Billing*. The platform takes in a continuous stream of smart-meter readings and one tariff file per day, and produces two outputs: a live dashboard showing the grid by zone, and a daily bill for each household. It uses a **Lambda architecture**, for the reasons given in Section 2. All the figures in the Results section were measured on the running system.

A power grid must balance supply and demand at every moment. Rooftop solar makes this harder, because the utility cannot control it: when a cloud passes over a zone, solar output can drop within a minute, and slower power stations have to make up the difference. Knowing the *renewable contribution of each zone right now* is therefore only useful if the information arrives quickly. At the same time, the billing department has to bill the same households from the same readings. A bill combines block tariffs, a fixed charge that depends on the household's tier, subsidy discounts, and a credit for solar energy exported to the grid.

1.2. Interpreted Business Requirements

The business question in the brief has two parts: *what is the grid load and renewable contribution in each zone right now*, and *what will each household's bill be once the daily tariff is applied*. These two parts serve two different users whose needs conflict (Table 1), and this conflict drives most of our architecture decision.

Table 1: The two consumers have opposing requirements.

Dimension	Grid operations	Billing
Question	Grid load and renewable mix by zone	Household bill once daily tariff is applied
Latency	Seconds	End of simulated day
A 2% error	Harmless; operators would act the same way	Legally actionable
Late data	May be ignored	Must be included
Restatement (re-billing a past day)	Never	Months later, on demand
Auditability	Not required	Every bill must be traceable to its readings

The words *once the daily tariff is applied* mean that the final bill depends on data that does not exist until the day has ended. This is part of the business process, so no streaming technique can avoid it. The outputs suggested in the brief map onto our system as follows. Real-time grid load by zone is served by the API endpoint `GET /api/v1/zones/load` and shown on the dashboard's Live grid (Section 7.1). The low-renewable alert is the `LowRenewableContribution` rule, supported by four further rules that watch the health of the pipeline (Section 6). The daily billing and solar report is available through the API, on the dashboard, and as a Markdown file written for every closed day (Section 7.2).

1.3. Functional and Non-Functional Requirements

F1. Ingest continuous meter readings, and one tariff file (plus a weather file) per simulated day.

- F2.** Keep every raw reading permanently and unchanged, so it can be processed again.
- F3.** Calculate each zone’s load and renewable contribution over short time windows.
- F4.** Compute per-household daily bills with solar netting, block tariffs, fixed charges, subsidy and export credit.
- F5.** Provide a live dashboard, a daily report, and alerts when values cross a threshold.

Table 2: Non-functional requirements and the measured outcome (Section 7).

ID	Requirement	Rationale	Measured
N1	Speed-layer latency below 60 s	Operators must react within a minute	Mean 6.3 s; none over 60 s
N2	Batch done within 10 real min of day close	Must fit in a live demonstration	Billing workflow ran in 59 s
N3	No data loss on a single-component failure	Kafka keeps messages for days; a permanent raw copy is kept	0 lost after killing either Spark job
N4	Past days can be re-billed at any time	Disputes outlast Kafka’s retention	Day re-billed from the raw copy
N5	Reproducible from a clean clone	Assessment requirement	One command (Appendix B)
N6	Every pipeline stage observable	Failures must be detectable	10 metrics, 5 alerts, 3 dashboards

1.4. Simulated Clock

Demonstrating a full real day would take far too long, so the system runs on a faster clock: **one simulated day passes in 5 real minutes**, which is 288 times faster than real time. Simulated time starts at 2026-01-01 00:00 UTC when the system starts, so the first day is incomplete and 2026-01-02 is the first full day. Each of the 50 meters, spread over 5 zones, sends a reading every 2 real seconds. This gives 150 readings per meter per simulated day, or about 7,500 readings in total. All data is grouped by simulated time, and only latency is measured with the real clock. A single module converts between the two clocks, and every duration in the configuration is labelled as either simulated or real, so they cannot be confused.

2. Architecture Decision: Lambda versus Kappa

Lambda [1] processes the same input data along two paths. A batch layer periodically recalculates exact results from all the stored data, a speed layer calculates approximate results from recent data as it arrives, and a serving layer combines the two. **Kappa** [2] uses a single streaming path and fixes mistakes by re-reading old data through the same code. **We chose Lambda.** Table 3 compares the two against the four criteria named in the marking rubric: latency, replay, cost and consistency.

The rejected alternative. Kappa would also have worked here. Tariffs would be stored in a Kafka topic that keeps only the latest tariff for each household, a single Spark streaming job would join them to the readings, and corrections would be made by re-reading old messages into a new table. Its main advantages are a single codebase, so the two paths can never disagree, one job to run, and one design to understand. We rejected it for three reasons. First, bills may need to be recalculated months later, but Kafka keeps messages for only days, so Kappa’s way of correcting results would not

Table 3: Architecture evaluation. Replay and consistency decide it.

Criterion	Analysis	Verdict
Latency	The two users need answers at very different speeds: seconds for operations, hours for billing. A single path can only be tuned for one of them. If it waits long for late data, the operations view is late; if it does not wait, bills are wrong.	Lambda
Replay	Billing disputes and tariff corrections can arrive <i>months</i> later, but Kafka’s <i>retention</i> (how long it keeps messages) is only <i>days</i> . Kappa fixes results by re-reading old messages from Kafka, which is impossible once they have expired. Lambda’s batch layer re-reads from object storage, which keeps data indefinitely.	Lambda
Cost	Lambda pays for two sets of code and a burst of computing once a day. Kappa pays for either very long retention or large amounts of per-household state held for months, a cost that grows with the number of households. At our scale both are cheap.	Lambda, narrowly
Consistency	Billing needs a permanent, auditable record that does not depend on the message system, so that every bill can be traced to its readings. Lambda’s <i>master dataset</i> , a permanent copy of every raw reading, provides this. In Kappa the only record is Kafka’s log, which is deleted according to a retention policy.	Lambda, decisively

be available when billing needs it. Second, a financial audit trail should not depend on how long a message system is configured to keep data. Third, holding billing state for every household for months is more expensive and more fragile than running one bounded job each night.

Lambda’s cost and how we contain it. With Lambda, the same billing logic exists in two places, and the two copies can drift apart without anyone noticing. For example, if the speed layer ended the first tariff block at 60 kWh and the batch layer ended it at 61 kWh, households near that boundary would see two different bills. To prevent this, all the netting, tariff, rounding, key and validation logic lives in one shared module, which only performs calculations, is fully tested, and is used by both layers. The logic still has to exist in two forms, because the API calculates estimates without Spark while the Spark jobs need Spark’s own expressions. To prove that the two forms agree, a consistency test runs both on 5,000 random inputs, and on values either side of every tariff block boundary, and checks that they produce the same bill to the cent. A difference in the logic would therefore fail a test. Differences in the inputs, such as a stale tariff or dropped late readings, are expected, and we measure them while the system runs (Section 7.4).

What would reverse the decision. Lambda is only justified while the stored raw data has to outlive Kafka’s retention. If Kafka’s tiered storage, or a table format such as Apache Iceberg or Delta Lake, gave the log unlimited and queryable history, Kappa’s single codebase would become the better choice.

3. Solution Architecture

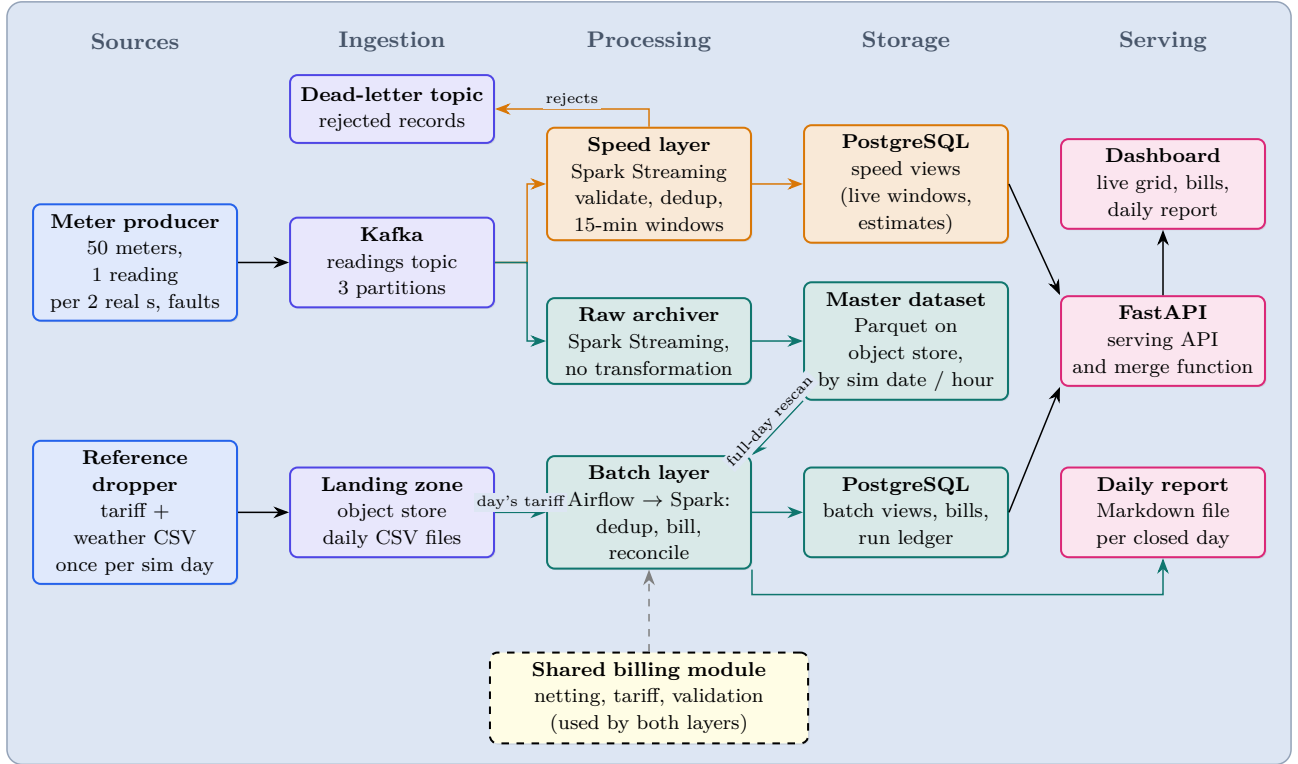


Figure 1: End-to-end architecture across the ingestion, processing, storage and serving layers. Box colours show the layer (blue for sources, indigo for ingestion, pink for serving). Amber marks the speed path, which processes data and then stores it; teal marks the batch path, which stores data and then processes it. Both paths read the same Kafka topic independently, each keeping track of its own position. The dashed shared billing module is used by both processing layers.

The three views. The *master dataset* contains every raw reading, unchanged, stored as Parquet files (a compressed, column-oriented format) on object storage and organised by simulated date and hour. It is never modified, and the archiver does not filter anything out, so any bug can be fixed later by recalculating from it. The *speed view* holds 15-minute (simulated) windows for each zone, with consumption, solar generation, renewable ratio and active meters, together with a running provisional bill for each household. That bill uses *yesterday's* tariff, because today's tariff does not exist yet. The *batch view* is built when a day's tariff file arrives: the whole day is read, validated, cleaned of duplicates, joined with that day's tariff and billed, and the result is then compared with the speed view.

The merge function. When a household's bill is requested for a day, the API checks the *run ledger*, a table recording every billing run. If the batch layer has already billed that day, the API returns the final bill; otherwise it returns the speed layer's provisional estimate (Figure 2). The batch bill always takes priority, because it includes the late readings that the speed layer missed and uses the correct tariff. Every response states which layer produced it and which day's tariff was used.

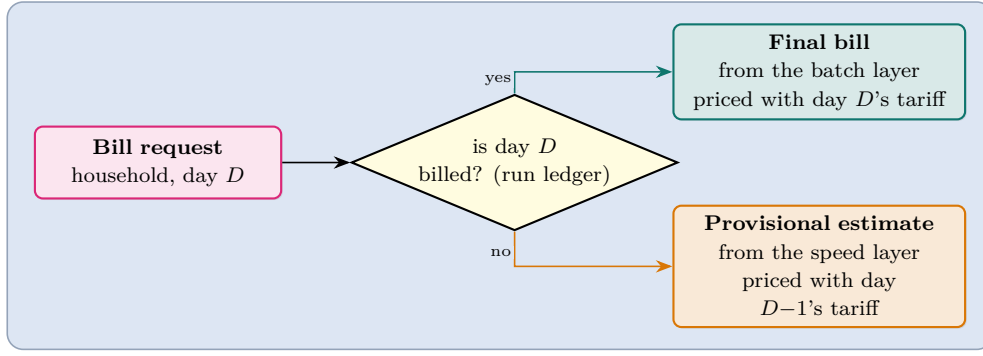


Figure 2: The merge function: the API checks the run ledger and returns either the final bill or the provisional estimate.

Key transformations. Readings are grouped into time windows by **event time**, which is when the meter took the reading, rather than when the reading arrived, because readings can arrive out of order. A *watermark* sets how long the speed layer waits for late readings before it closes a window. **Solar netting** is applied before the tariff:

$$self = \min(solar, consumption), \quad import = consumption - self, \quad export = solar - self,$$

The bill is then $total = energy_charge + fixed_charge - subsidy - export_credit$, where the energy charge prices the imported energy block by block, and money is rounded half-up to two decimal places in exactly the same way in both layers. Because Kafka may deliver the same reading twice, both layers **remove duplicates** using the meter ID and reading time, a rule defined once in the shared module. Kafka messages are keyed by household, so each household’s readings stay in order.

4. Technology Stack and Justification

Table 4 lists each technology we chose, the requirement it meets and the alternative we rejected. The paragraphs below explain each choice.

Kafka. Kafka [3] is a log rather than a queue: reading a message does not delete it. This means a bug in the speed layer can be fixed by re-reading old messages, the producer keeps working when a reader is down, and the speed layer and the archiver can read the same topic independently. We confirmed that stopping one reader did not affect the other. The topic has three partitions, which limits how many readers can work in parallel, and Kafka’s default seven-day retention is the limitation discussed in Section 2. RabbitMQ was rejected because it deletes a message once it has been read, so it supports neither re-reading nor independent readers.

Spark. Spark [4] uses the same programming interface for batch and streaming jobs, so a single billing module can serve both layers. This directly addresses Lambda’s biggest risk. Spark also supports event-time windows and watermarks natively. Storm offers lower latency, but it has no event-time windows and no batch interface, and since we only need answers within seconds, its extra speed would not help us. Flink is a stronger pure streaming engine, but it does not offer an equally simple way to share batch code in Python, and Spark already meets our latency target (6.3s against a 60s limit).

Parquet on object storage. Lambda needs every raw reading to be kept. If a bill is wrong because of a logic error, such as using $>$ instead of $\geq$ at a block boundary, the fix is to correct the code and

Table 4: Technology selections and the constraint each satisfies.

Layer	Selected	Constraint satisfied	Rejected
Ingestion	Apache Kafka 3 (KRaft)	Data can be re-read, which Lambda needs; two readers can consume the same stream independently	RabbitMQ; direct DB writes
Stream & batch	Spark 3.5 Structured Streaming and batch	The same API for both layers allows one shared billing module; built-in event-time windows	Storm; Flink
Master dataset	Parquet on S3-compatible object store	Compressed column format that is never modified; a full day can be re-read quickly; storage is separate from computing	HDFS; raw CSV/JSON
Serving store	PostgreSQL 16	Joins, safe transactions for money and fast lookups; our data volume after aggregation is small	Cassandra; MongoDB
Orchestration	Apache Airflow 3	Waits for files, retries failures, keeps a run history and can re-run a single day (used for re-billing)	cron
Observability	Prometheus, Alertmanager, Grafana	Collecting metrics from containers is simple; alert rules are stored and tested with the code	Custom log parsing
Serving API	FastAPI	Generates API documentation automatically and validates every request	Flask
Deployment	Docker Compose	The whole system starts with one command, without a cloud account	Kubernetes

recalculate from the raw readings. Parquet stores data by column, compresses it well, keeps proper data types and lets queries skip data they do not need, and organising the files by date and hour means the daily job reads only its own day. We use a local S3-compatible object store so that markers do not need a cloud account; because the code uses only the standard S3 interface, moving to AWS S3 would only require a configuration change. Since MinIO stopped publishing its images, the system runs `pgsty/silo`, a MinIO-compatible replacement. HDFS was rejected as too heavy for a system of this size.

PostgreSQL. Our queries look up a household’s bill for a date, total the recent readings for each zone, and join bills with the run ledger and household details, all of which return small, relational results. PostgreSQL supports joins, transactions (so a billing run is saved completely or not at all), JSON columns for the block breakdown, and writes that can safely be repeated. Cassandra is designed for much higher write volumes than ours, and it does not support joins.

Airflow. Airflow [5] schedules work by real time, so “once per simulated day” cannot be written as a normal schedule. Instead, a small workflow (a *DAG*) called `tariff_watcher` checks the landing zone every real minute and starts the `daily_billing` workflow once for each new tariff file. Each run is named after the file’s date (`billing__2026-01-02`), so the same day is never billed twice by accident, and re-billing a day simply starts another run for that date (`billing__2026-01-02__r1`). The workflows only start Spark jobs and check their results; they contain no billing logic. Cron was rejected because it has no task dependencies, retries, run history or re-runs.

Prometheus, Alertmanager and Grafana. Each service publishes its metrics, and Prometheus [6] collects (*scrapes*) them every 15 seconds. Batch jobs finish too quickly to be collected this way, so

they push their metrics to a Pushgateway instead. The alert rules are stored in the repository and tested with `promtool`, and the Grafana dashboards are created automatically from files.

5. Implementation

5.1. Deployment

The system runs as eighteen containers under Docker Compose, started in four tiers, where each tier waits until the previous one is healthy (Figure 3). A single command starts and checks everything (`voltstream.ps1` run on Windows, `make up` on Linux). The bootstrap containers create the Kafka topics, database tables and storage buckets, and then exit. The whole system uses 5.3–6.4 GiB of memory, so the README recommends giving Docker at least 8 GB.

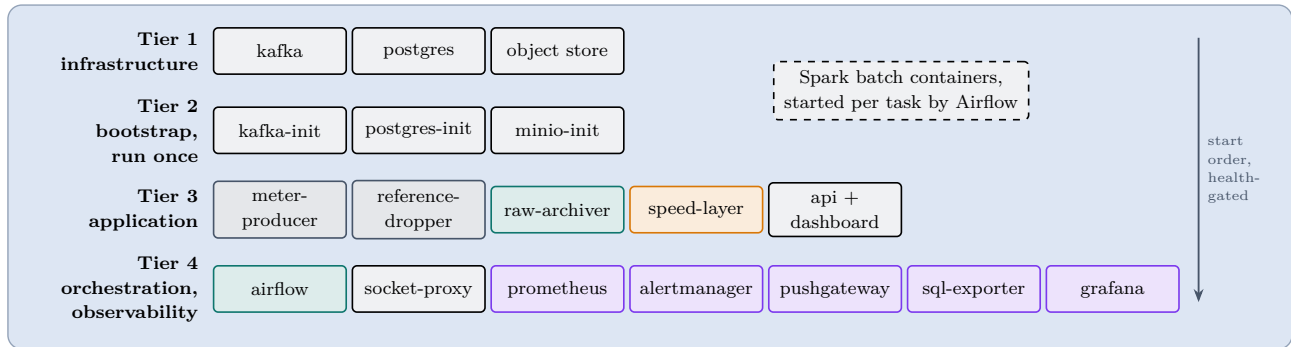


Figure 3: Deployment: 18 containers in four tiers, each started once the previous tier is healthy (15 run continuously and 3 run once at start-up). Billing, rollup and reconciliation run in short-lived Spark containers started by Airflow.

5.2. Data Sources and Ingestion

Two Python programs simulate the data sources. The **meter producer** calculates each household’s consumption and solar generation for the current simulated time (solar is zero between 18:00 and 06:00), adds random variation, and publishes a reading to the Kafka topic `meter.readings` every 2 seconds:

Listing 1: A meter reading event, keyed by `household_id`.

```
{
  "schema_version": "1.0",
  "event_id": "uuid4",
  "trace_id": "uuid4",
  "meter_id": "MTR-0042",
  "household_id": "HH-0042",
  "grid_zone": "ZONE-C",
  "event_ts": "2026-01-02T14:15:00Z", // simulated time; drives all windowing
  "consumption_kwh": 0.412,
  "solar_generation_kwh": 0.180,
  "voltage": 232.4,
  "producer_id": "sim-01"
}
```

The **reference dropper** writes two files to the landing zone for each simulated day: a tariff file (for each household: tier, subsidy flag, three block rates, fixed charge, export rate and effective date) and a weather file. Tariff rates change by ± 0.50 each day, so using yesterday’s tariff makes a visible difference. The formats of the readings, tariffs and weather records are fixed as schemas, and every tariff row is checked before it is used.

To test how robust the pipeline is, the producer deliberately introduces faults at configurable rates (Appendix A): duplicate readings, readings with missing fields or negative values, readings from unknown households, and *meter dropouts*, where a meter loses its connection, stores its readings and sends them late once it reconnects. *Reordering*, where readings arrive slightly out of order, is introduced separately. Invalid readings are sent to a dead-letter topic (`meter.readings.dlq`) and

recorded, with the reason, in the `rejected_records` table. A test checks that when every fault rate is set to zero, the output is identical to the input.

5.3. Processing

Raw archiver. This Spark streaming job writes every Kafka message to Parquet without changing it, organised by simulated date and hour, and after a restart it continues from where it stopped. It guarantees that *no data is lost*, but not that each reading is written exactly once: Spark saves the files and its reading position in two separate steps, so a crash between them causes a small range to be read again. After deliberately killing the archiver, we found 334 duplicate rows out of 14,012 and no missing readings; the batch layer removes such duplicates.

Speed layer. A second, independent Spark streaming job checks each reading with the shared validation rules, removes duplicates, and groups the readings into 15-minute (simulated) windows for each zone, using a 30-minute (simulated) watermark. It also keeps a running total for each household, and writes its results to PostgreSQL every 10 real seconds; this interval is called the *trigger*. The provisional bill is calculated with the shared billing logic and yesterday’s tariff.

Running the clock 288 times faster has two side effects here. First, 10 real seconds equals 48 simulated minutes, which is longer than the 30-minute watermark, so in practice it is the trigger interval, not the watermark, that decides which late readings are dropped (see Section 7.4). Second, results are published after every trigger, so the watermark does not delay them; it only decides when old windows are closed.

Batch layer. The `daily_billing` workflow has eight tasks, which run in this order: wait for the day’s tariff file; wait a further 90 real seconds for late readings; run the Spark billing job, which reads the day’s Parquet files, validates them, removes duplicates, joins the tariff, applies the shared netting and tariff logic, writes the bills and records the run in the run ledger; check that the number of bills is correct; check that every bill adds up; calculate exact daily totals for each zone and check that they match the household totals; reconcile the two layers; and write the daily report.

Reconciliation. For each household, the difference between the speed estimate and the final bill is split into two parts. The *tariff effect* is the part caused by using yesterday’s prices: the speed layer’s energy priced at today’s tariff, minus its estimate. The *data effect* is the rest: the final bill minus that re-priced figure, which reflects readings the speed layer missed. The two parts always add up to the total difference.

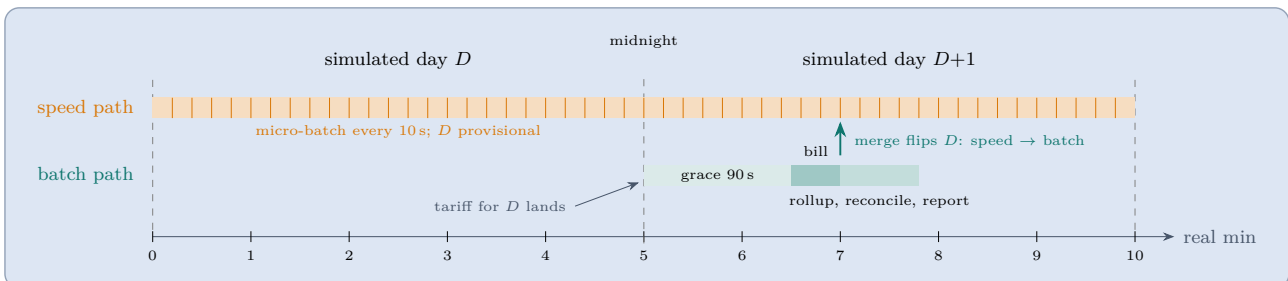


Figure 4: Timing of one simulated day. The two paths run at the same time and neither waits for the other. If either fails, no data is lost, because the raw readings remain in Kafka and in Parquet. The batch timings are those measured in the reference run.

5.4. Storage and Serving

The speed and batch results are stored in *separate tables* (Table 5), so neither layer overwrites the other and every row shows which layer produced it. The FastAPI service provides endpoints for zones, households, bills (using the merge function), the difference between estimate and final bill, daily reports, alert status and health checks. It generates its own documentation page at `/docs` and also serves the dashboard.

Table 5: Serving-layer tables in PostgreSQL.

Table	Written by	Purpose
<code>zone_metrics_rt</code>	Speed	Per-zone windowed load and renewable ratio
<code>household_running_rt</code>	Speed	Provisional running total and estimated bill
<code>household_bill_daily</code>	Batch	Current final bill with block breakdown
<code>household_bill_history</code>	Batch	Every billing run’s bills, append-only (audit trail)
<code>zone_metrics_daily</code>	Batch	Exact daily zone rollup
<code>pipeline_runs</code>	Batch	Run ledger; the finalisation flag the merge reads
<code>reconciliation_daily</code>	Batch	Speed-vs-batch gap, split into tariff and data effects
<code>rejected_records</code>	Both	Rejected records with reason and <code>trace_id</code>

5.5. Code Quality and Testing

The code is a typed Python 3.11 package with one folder per layer (`simulators`, `streaming`, `batch`, `storage` and `api`) plus the shared billing module. All settings are kept in one configuration file, which is checked when the system starts. There are 281 automated tests (unit, property-based, consistency and integration), tests for the alert rules using `promtool`, and a test that fails if the alert thresholds or metric names no longer match the code. On every push, GitHub Actions checks code style and types, runs the tests, requires full test coverage of the shared billing module, and checks the alert rules.

6. Observability Design

Good observability tells an operator three things: whether data is flowing, whether it is correct, and, if not, where it is stuck. Figure 5 shows how each part of the pipeline is monitored.

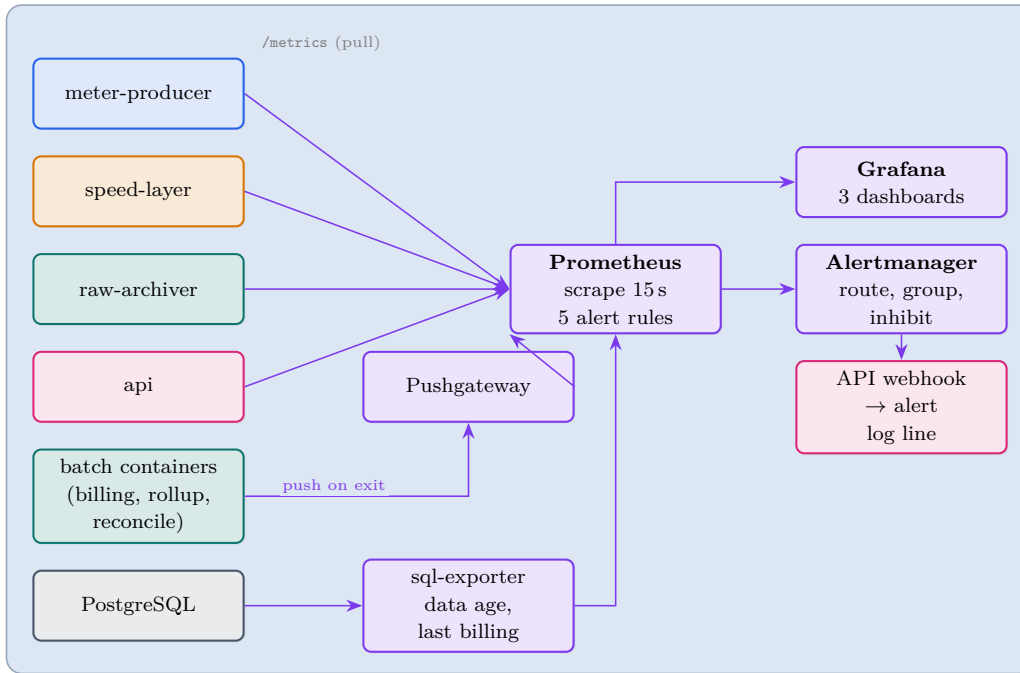


Figure 5: Observability: services are scraped, batch containers push, **sql-exporter** turns two database facts into metrics, and alerts return to the API as log lines and to the dashboard banner. Every service also writes JSON logs.

Structured logging. Every service writes its log as one JSON object per line, always with the same fields, so a single reading can be followed across containers with one search:

Listing 2: A speed-layer log line.

```
{
  "ts": "2026-09-27T13:35:25.992Z",
  "level": "INFO",
  "service": "speed-layer",
  "stage": "speed-zone",
  "trace_id": "...",
  "sim_date": "2026-01-01",
  "msg": "zone windows upserted",
  "batch_id": 42,
  "rows_out": 15
}
```

The **service** and **stage** fields show which part of the pipeline wrote each line: ingestion (**producer**), processing (**speed-zone**, **archive**, **billing**) or serving (**api**). Alertmanager also sends its notifications to an API endpoint, which logs them with **stage: alert**, so alerts appear in the same log stream as everything else.

Metrics. Table 6 lists what is measured and why. Latency is measured from the moment a reading enters Kafka to the moment the view read by the dashboard is updated, so the 60-second requirement is checked rather than assumed. The **lambda_divergence** metric lets the system watch for Lambda’s main weakness, the two layers drifting apart. The two **pg_** metrics are read from the database for two reasons: the renewable ratio is zero every night, so an unchanging value cannot reveal a dead data feed, and only the run ledger knows when billing last succeeded.

Tracing. Full distributed tracing inside Spark was beyond the scope of this project, so we use **correlation IDs** instead. The producer gives each reading a **trace_id**, which travels with it through Kafka into the Parquet files, the dead-letter topic and every log line. The API does the same for requests, using an **X-Trace-Id** header. Searching for one ID therefore shows a reading’s full path and timing.

Alerts and health checks. There are five Prometheus alert rules (Table 9), stored in the repository, and a test checks that their thresholds match the configuration. Alertmanager handles critical and warning alerts separately, and it suppresses a zone’s **LowRenewableContribution** alert while

Table 6: Exported metrics (prefix `voltstream_`). The last two come from `sql-exporter`.

Metric	Type	Labels	Answers
<code>events_produced_total</code>	Counter	producer	Is the source alive?
<code>events_consumed_total</code>	Counter	layer	Is each consumer keeping up?
<code>records_rejected_total</code>	Counter	layer, reason	What bad data, and how much?
<code>e2e_latency_seconds</code>	Histogram	layer	Where does latency go?
<code>consumer_lag</code>	Gauge	layer, partition	Is a branch falling behind?
<code>zone_renewable_ratio</code>	Gauge	zone	Business metric; drives an alert
<code>batch_duration_seconds</code>	Histogram	job	Is the nightly job slowing?
<code>lambda_divergence</code>	Gauge	-	Do the two layers agree?
<code>pg_zone_data_age_seconds</code>	Gauge	zone	Has a zone stopped reporting?
<code>pg_billing_last_success_timestamp</code>	Gauge	-	When was a day last billed?

`MeterDataStale` is firing for that zone, because missing data would explain the missing solar. The API also provides health checks (`/health/live`, and `/health/ready`, which confirms that PostgreSQL and object storage are reachable) and an alert status endpoint (`/api/v1/alerts/status`). Grafana has three dashboards: pipeline health, grid operations and Lambda divergence.

7. Results

All results were measured on the complete system running under Docker Compose on one Windows laptop (16 CPUs, with about 7.5 GiB of memory available to Docker). The *reference run* on 27 September 2026 was the first uninterrupted run with both paths working, and the fault tests were run the following day. The system’s own end-to-end check (`voltstream.ps1 check`) reported 37 checks passed and none failed.

7.1. Real-Time Zone Monitoring

In 40 real seconds, the newest zone window advanced from 07:30 to 10:45 in simulated time, which is 3 hours 15 minutes and confirms the 288-times speed-up. The Live grid (Figure 6) requests `/api/v1/zones/load` every 5 seconds and shows the load, solar generation, renewable share and active meters for each zone. The renewable share falls to 0% every simulated night, which is when the `LowRenewableContribution` alert fires.

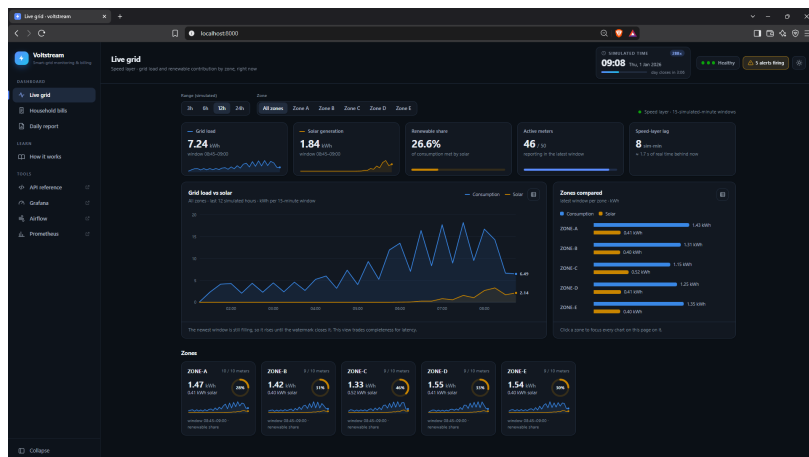
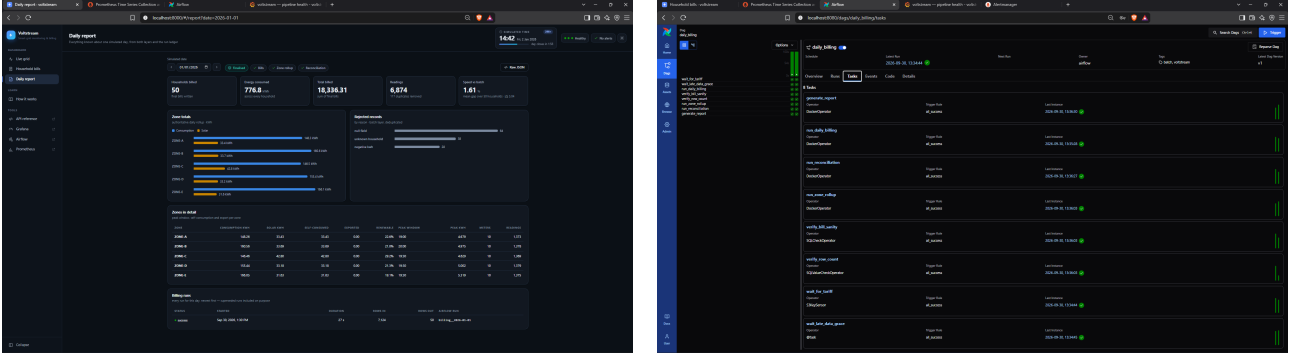


Figure 6: Live grid (speed layer): current grid load and renewable contribution by zone.

7.2. Daily Billing and Solar Report

For 2026-01-02, the first complete day, the batch layer read 7,029 readings and produced 50 bills, and the whole eight-task workflow took 59s. Every bill added up correctly. The last task of the workflow writes a Markdown report for the day, containing zone totals (consumption, solar, self-consumed, exported and renewable share), billing totals, rejected readings grouped by reason, the difference between the two layers, and the billing runs. The same information is available from `/api/v1/reports/daily` and on the dashboard's Daily report tab (Figure 7).



(a) Daily report for 2026-01-02.

(b) The eight-task `daily_billing` DAG.

Figure 7: The consolidated daily report and the DAG that produces it.

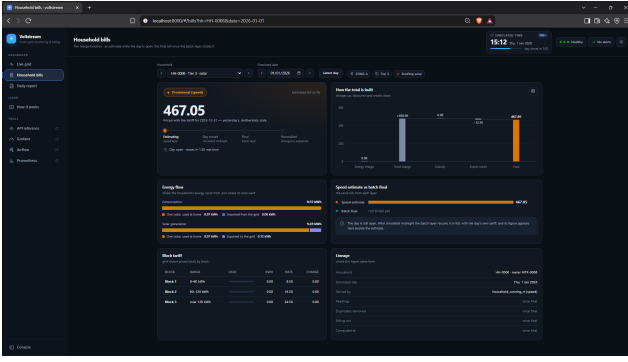
7.3. Provisional-to-Final Transition

We requested the bill for household HH-0001 for 2026-01-02 at three points: during the day, at the end of the day, and after the batch layer had billed it (Table 7, Figure 8).

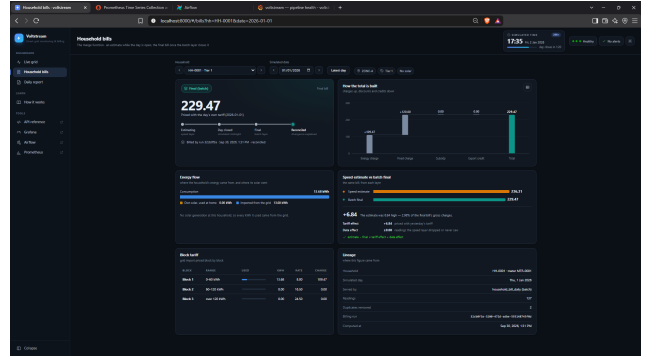
Table 7: The merge function switching from speed to batch (reference run).

	Mid-day	At day close	After billing
source / provisional	speed / true	speed / true	batch / false
Tariff used	2026-01-01	2026-01-01	2026-01-02
Energy	10.0665 kWh so far	-	13.4363 kWh (138 readings, 3 duplicates removed)
Total	200.53	227.49	234.21

The final bill was 6.72 (2.869%) higher than the estimate at the end of the day. Almost all of this difference came from the tariff: the 2026-01-02 tariff was higher than the one the speed layer had used. A repeat run showed the same pattern (final bill 237.38, a 2.907% difference, all of it due to the tariff).



(a) Before finalisation.



(b) After finalisation.

Figure 8: The merge function as the user sees it: the badge switches from provisional to final.

7.4. Speed-versus-Batch Reconciliation

For each household, the difference between the speed estimate and the final bill is split into two parts: the *tariff effect* (caused by using yesterday’s prices) and the *data effect* (caused by readings the speed layer missed). Table 8 shows the averages over all 50 households.

Table 8: How the gap between estimate and final bill splits, averaged over 50 households per day (day 1 is incomplete).

Day	Mean tariff effect	Mean data effect	Tariff share	Mean gap	Max gap	kWh missed by speed
2026-01-01	+5.14	−0.89	85.2%	1.570%	3.537%	0.861%
2026-01-02	−5.35	−0.61	89.7%	1.878%	4.450%	0.523%

The tariff effect appears for every household and changes sign each day, because the tariff goes up and down on alternate days. The data effect appears for only 10–12 households a day. These are households whose meters dropped out: their stored readings arrived after the watermark, so the speed layer missed them but the batch layer billed them.

On the second day the average gap was 1.88%, well under the 5% level at which the `LambdaDivergenceHigh` alert fires (Figure 9a).

One result surprised us. In the live 15-minute view, 1.16% of the day’s energy was missing. With dropouts turned off it was still 1.00%, so most of the loss comes from ordinary reordering. We had expected the watermark to absorb that. It did not, because at our time compression each 10-second batch covers 48 simulated minutes, longer than the 30-minute watermark.

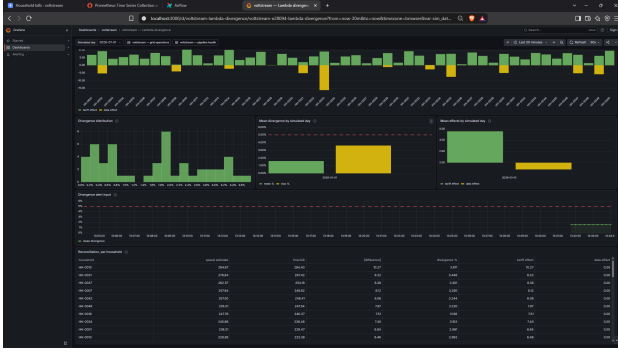
7.5. Observability in Operation

Each alert was tested by deliberately causing the fault it is meant to detect (`voltstream.ps1 faults`), confirming through Alertmanager that the alert fired, and then removing the fault (Table 9).

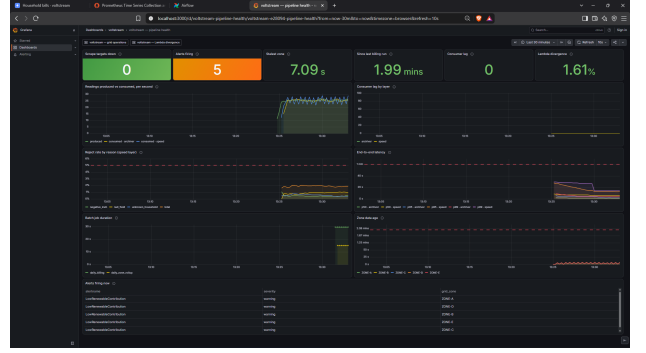
Rejected readings can be traced: the 53 rows in `rejected_records` matched the 53 messages in the dead-letter topic exactly, and the same `trace_id` was found in both places.

Table 9: Alert rules, what they detect, and measured behaviour under an induced fault.

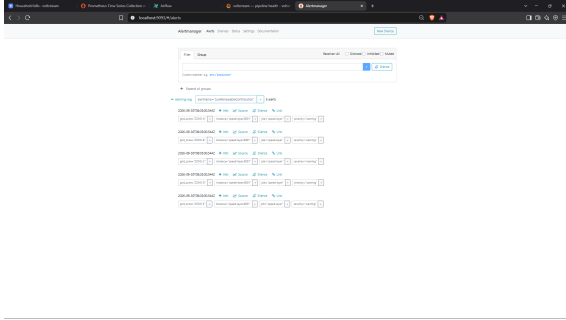
Rule (severity)	Condition	Induced fault and result
MeterDataStale (critical)	A zone’s newest window older than 2 real min	Producer stopped: fired after 173–195 s for all 5 zones; cleared 41–46 s after restart
LowRenewable- Contribution (warning)	Zone renewable ratio below 15% for 3 windows (the brief’s alert)	No fault needed: fired at dusk (19:40 simulated) for all zones and cleared at 10:00 simulated
HighRejectRate (warning)	Rejects above 5% of consumed records over 5 min	15% null fields injected: fired after 119 s at 11.6% (normal about 2%)
BatchSLAMiss (critical)	No successful billing within 10 real min of day close	Billing DAG paused: fired 21 s after due; cleared 6 s after billing resumed
LambdaDivergence- High (warning)	Mean speed-vs-batch gap above 5%	Not reached live (the run stalled on memory); fired when the threshold was lowered to 1%, on the 1.68% gap of the latest reconciled day at that time



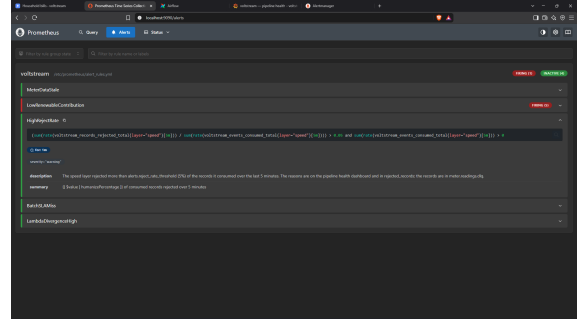
(a) Lambda divergence dashboard.



(b) Pipeline health while MeterDataStale fires.



(c) Alertmanager, all five zones.



(d) Prometheus rule state.

Figure 9: Monitoring and alerting under induced faults.

7.6. Resilience and Re-billing

Crashing each path. We stopped the raw archiver abruptly (SIGKILL) for 20 s. No readings were lost and there were no gaps in Kafka; the 334 duplicates this caused were removed by the batch layer. We then stopped the speed layer for 60 s. During that time the live view was 56 s out of date, while the archiver continued to store readings (1,526 of them). After restarting, the speed layer caught up within 22 s, the previous day’s final bill stayed at 237.38, and the day of the crash was still billed in full. In other words, when the speed layer fails, the live view is briefly out of date, but no bill is affected.

Re-billing a day. The `voltstream.ps1 backfill` script deliberately corrupted the tariff for 2026-01-02 (adding 45.00 to the block 1 rate), re-billed the day, restored the correct file and re-billed it again. The day’s total went from 18,737.41 to 44,876.57 and then back to exactly 18,737.41. The run ledger shows the three runs as **superseded**, **superseded** and **success**, each linked to its own Airflow run, and `household_bill_history` keeps all three versions of every bill. Bills are always recalculated from the stored raw data, never edited directly.

7.7. Measured Latency

Table 10: End-to-end latency over 20 minutes of normal running (including billing runs and a producer outage); API reads timed with 40 requests each. p50 and p95 are the times that 50% and 95% of readings stayed under.

Stage	Design estimate	Measured
Kafka → zone view (speed)	about 40 s	mean 6.3 s , p50 5.1 s, p95 14.3 s; none of about 2,300 over 60 s
Kafka → Parquet (archiver)	-	mean 3.0 s, p50 1.0 s, p95 13.3 s
Trigger wait	about 10 s	0–10 s, 5 s on average: the main term
Watermark wait	about 30 s	0 s, because results are published every batch (our estimate was wrong)
API reads (zones, bill)	20–50 ms	p50 14.7 ms and 8.7 ms

The speed path met the 60-second requirement for every reading, usually within about 5 s. The slowest batches happened during billing runs, when the batch and streaming Spark jobs were competing for the same laptop.

8. Discussion, Limitations and Production Scale

Was Lambda the right choice? We think so. The provisional bill was always going to differ from the final one, because it uses yesterday’s tariff, and the batch layer replaced it with the correct bill about 2.5 minutes after each day closed. Re-billing a past day tested the main reason we rejected Kappa: we recomputed the day from the stored Parquet files, without relying on Kafka still holding the data. Lambda did cost more, as expected (two jobs, an orchestrator and a reconciliation step), but because both layers share one billing module, every difference between them came from their inputs, never from the billing logic.

Limitations.

- **Time compression** speeds up simulated time but not the computer’s processing time, so the watermark settings are not realistic. This is why the trigger interval ended up longer than the watermark, and why the live view misses about 1% of the energy. Setting the watermark longer than one trigger interval would fix this.
- **Simulation:** meters send the energy used in each interval rather than a running total; consumption, solar and tariffs are synthetic; and 50 households is far fewer than a real utility serves. The weather file is produced but is not used for billing or solar generation.
- **Small files:** one simulated day produced 245 Parquet files averaging only 10.2 kB, far smaller than Parquet works best with. A production system would need to merge them regularly, or use a table format that does so.
- **Single points of failure:** there is one Kafka broker, one Spark node and one PostgreSQL

server, no schema registry, and passwords are stored as local defaults in `.env`.

- **Resources:** the system needs about 9 GB of memory. On the demonstration laptop the divergence test ran out of memory, so that alert was demonstrated with a lower threshold.
- **Monitoring gaps:** if `sql-exporter` stops, two alerts stay silent instead of firing, and the Pushgateway keeps showing the last divergence value indefinitely. Tracing uses correlation IDs rather than full distributed tracing.

Production scale. In production we would use Iceberg or Delta Lake on top of the raw Parquet files (for transactions, automatic merging of small files and access to past versions), a schema registry with Avro or Protobuf, change data capture instead of daily tariff files, replicated Kafka, Spark on Kubernetes, watermarks based on measured delays, and alerts for when the exporters themselves stop. PostgreSQL would be enough for up to a few million summary rows per day; beyond that, a column-oriented database such as ClickHouse would fit better. Iceberg or Kafka tiered storage would also remove the retention limit that decided Section 2, at which point Kappa would become the better choice.

9. Conclusion

Voltstream uses one stream of meter readings to serve two very different needs. Grid operators need the load and renewable share of each zone within seconds, while billing needs an exact, auditable bill that cannot be produced until the daily tariff arrives. We chose Lambda because bills must be recalculable long after Kafka has deleted the original messages, and we limited Lambda’s main risk by sharing all billing logic through one tested module. The system is built with Kafka, Spark, Parquet on object storage, PostgreSQL, Airflow and FastAPI, and starts with a single command. On the running system, the speed path delivered zone metrics in about 6 seconds; the batch layer finalised all 50 bills about 2.5 minutes after each day closed; each bill switched from provisional to final as designed; a corrupted day was re-billed with its history kept; neither crash lost any data; and all five alerts fired when their faults were introduced. The main limitation is the compressed clock, which distorts the timing of the streaming path. The architecture decision itself would only change if Kafka’s data could be kept and queried indefinitely.

References

- [1] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Manning Publications, 2015.
- [2] J. Kreps, “Questioning the Lambda Architecture,” O’Reilly Radar, 2014.
- [3] Apache Software Foundation, *Apache Kafka Documentation*. <https://kafka.apache.org/documentation/>
- [4] Apache Software Foundation, *Structured Streaming Programming Guide*. <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- [5] Apache Software Foundation, *Apache Airflow Documentation*. <https://airflow.apache.org/docs/>
- [6] Prometheus Authors, *Prometheus Documentation*. <https://prometheus.io/docs/>

Appendix A: Assumptions and Simplifications

1. Simulated time runs 288 times faster than real time: one simulated day is five real minutes, starting at 2026-01-01 00:00 UTC when the stack starts. All timestamps are UTC.
2. Meters report the energy used in each interval (kWh), not a running total.
3. Fifty households in five fixed grid zones. Kafka retention is the default seven days.
4. Tariff values are synthetic (Table 11); only the block boundaries are configuration, and every rate comes from the daily tariff file.
5. The fault rates (Table 12) were chosen to test validation and alerting, not to reflect real failure rates.
6. The weather file satisfies the brief’s second daily source but is not used in billing.

Table 11: Synthetic tariff (configuration defaults).

Parameter	Value
Block boundaries	0–60, 60–120, >120 kWh
Block rates	8.00, 16.50, 24.50 per kWh
Daily change	block 1 rate $\pm$ 0.50
Fixed charge	120 / 240 / 480 by tier
Subsidy	25%
Export rate	18.00 per kWh

Table 12: Fault-injection rates (per reading).

Fault	Rate	Tests
Duplicate	2.0%	Dedup
Out of order	3.0%	Watermark
Null field	1.0%	Validation
Negative kWh	0.5%	Validation
Unknown household	0.5%	Validation
Meter dropout	0.2%/tick	Late data

Appendix B: Reproducing the Results

Needs Docker Desktop (at least 8 GB memory, 4+ CPUs) and Git. Full steps are in the README.

Listing 3: From a clean clone to every result (Windows; each has a `make` equivalent on Linux).

```
git clone https://github.com/Malee-Fonseka/voltstream.git && cd voltstream
.\scripts\voltstream.ps1 run # build and start all containers
.\scripts\voltstream.ps1 check # end-to-end verification
.\scripts\voltstream.ps1 demo # provisional -> final -> delta
.\scripts\voltstream.ps1 faults # all five alert drills
.\scripts\voltstream.ps1 killtest # crash and recover the speed layer
.\scripts\voltstream.ps1 backfill -Date 2026-01-02 # restate a day
```

Appendix C: Statement of Individual Contributions

All three members contributed equally to the project. The work was split by implementation phase so that each member owned one part of the pipeline end to end, and work was merged into the main branch through pull requests. The architecture decision was discussed and agreed by the whole group.

Registration	Name	Contribution	Share
EG/2021/4511	Fernando S.A.U.	Raw archiver and master dataset; speed layer; storage layer and serving API; batch billing layer and its Airflow DAGs; merge function and daily report generator; archiver restart and batch integration tests; architecture diagrams and this report.	33.3 %
EG/2021/4516	Fonseka C. M.	Project set-up and task plan; architecture design and decision log (Lambda vs Kappa, D1–D9); reconciliation; observability stack (Prometheus, Alertmanager, Grafana, sql-exporter); fault-injection, demo and restatement scripts; dashboard; CI and reproducibility; README, runbook and debugging.	33.3 %
EG/2021/4525	Gunarathna P.M.S.C.B.	Cross-cutting modules (configuration, simulated clock, structured logging, metrics registry); data contracts and JSON Schema export; infrastructure bring-up (Kafka topics, PostgreSQL schema, object-store buckets); the shared core/ module (netting, tariff, validation, keys) with its property and consistency tests; meter and reference simulators with fault injection.	33.3 %